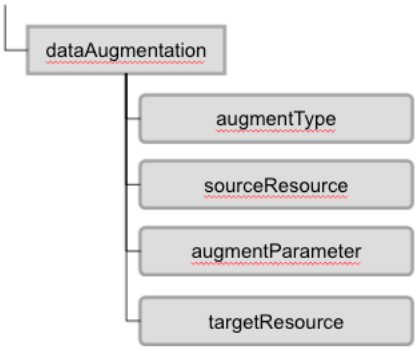
Data Augmentation 추가

Data Augmentation resource tree



resource: https://onem2m.org/using-onem2m/developers/api#n2a

resource: https://github.com/IoTKETI/oneM2M-API-Testing/blob/master/REST_API_for_Mobius_Rel2.pdf

Data Augmentation resource tree
1. Modules
    1.1 app.js
    1.2 resource.js
    1.3 responder.js
    1.4 sql_action.js
    1.5 da.js
2. app.js
3. resource.js
    3.1 require
    3.2 ty_list
    3.3 attribute
    3.4 create_action()
    3.5 build_resource()
    3.6 update_action()
4. responder.js
    4.1 typeRsrc
    4.2 attrLname
    4.3 attrSname
5. sql_action.js
    5.1 insert_da
    5.2 update_da
6. da.js
7. Create table
8. python test

1. Modules

이 문서는 Mobius-YT server에 새로운 리소스 타입인 'dataAugmentaion'을 추가하는 내용을 정리했습니다. 새로운 리소스 타입을 Mobius 코드에 추가할 때는 4개의 파일을 수정하고, 1개의 파일을 새롭게 만들어야 합니다. 각 파일에 대한 자세한 설명은 아래 문서에서 확인할 수 있습니다.

https://github.com/IoTKETI/Mobius/blob/master/doc/Installation

1.1 app.js

Mobius-YT server를 실행하는 main code입니다. 트래픽 플로우를 실시간으로 평가하는 flow router 역할을 합니다. 받은 패킷의 초기 핸들 처리, 파싱, request 유효성 검사를 진행하고 옳바른 리퀘스트라면 resource.js에 전달합니다.

1.2 resource.js

리소스의 CREATE, RETRIEVE, UPDATE, DELETE, NOTIFY, DISCOVERY 기능을 처리합니다. 파싱된 request URI와 request body를 app.js로 부터 전달 받아 그에 맞는 동작을 수행합니다. 데이터를 가공하고 sql_action.js모듈을 사용해 mysql DB에 연결합니다.

1.3 responder.js

app.js와 resource.js 모듈의 결과값을 받아 처리하고 XML 혹은 JSON으로 response를 생성합니다.

1.4 sql_action.js

가공된 데이터를 resource.js로 부터 전달 받아 sql query를 생성하고 DB와 연동합니다.

1.5 da.js

dataAugmentaion 리소스를 담당하는

2. app.js

```
function lookup_create(request, response, callback) {
  ...
  else if ((request.ty == 50) && (parentObj.ty == 5 || parentObj.ty == 2 || parentObj.ty == 3)) { // dataAugment
  }
  ...
}
```

`lookup_create()`에서 `request.ty`와 `parentObj.ty`를 정의해줘야합니다.

dataAugment 리소스의 `request.ty`를 50으로 정의했습니다.

해당 값은 리소스 생성 request의 `Content-Type`에 명시해주는 값입니다.

다음과 같은 리소스들이 dataAugment 리소스의 parent가 될 수 있습니다.

- 5 == csebase
- 2 == ae
- 3 == container

3. resource.js

3.1 require

```
var da = require('./da.js')
```

da.js 파일을 require()로 추가해줍니다.

3.2 ty_list

```
global.ty_list = ['1', '2', '3', '4', '5', '9', '10', '13', '14', '16', '17', '23', '24', '27', '28', '29', '30', '38', '39', '50', '91', '
```

`global.ty_list` 에 dataAugment리소스에 해당하는 `50` 을 추가해줍니다.

3.3 attribute

1) Abbreviations

NP (Not Present): NP mark indicates attributes that don't necessarily be present in the resource representation for the corresponding request via API.

M (Mandatory): M mark indicates attributes shall be present in the resource representation either for the corresponding request via API or the responses provided by the specific IoT platform.

O (Optional): O mark indicates attributes that are allowed to be present in the resource representation for the corresponding request for API but will not affect the request result if absent.

resource.js에는 각 리소스에 대한 creaet, update 동작시 필요한 attribute를 정의해줘야 합니다.

각 리소스마다 세 종류(NP, M, O)의 attribute를 정의해줘야 합니다. M attribute에는 리소스의 핵심 attribute를, 나머지 NP와 O attribute는 다른 리소스들과 통일 시켜주면 됩니다.

M attribute들은 request에 필수적인 attribute를 뜻하고, NP와 O attribute들은 해당 리소스를 represent할때 필수적이지 않거나, 다른 module에서 추가하기 때문에 request에 명시할 필요가 없는 attribute들입니다.

```
create_np_attr_list.da = ['ty', 'ri', 'pi', 'ct', 'lt', 'st'];
```

Create NP (Not Present) attribute

```
create_m_attr_list.da = ['augty', 'srsrc', 'augprm', 'trgrsrc'];
```

Create M (Madatory) attribute

```
create_opt_attr_list.da = ['rn', 'acpi', 'et', 'lbl', 'cr'];
```

Create O (Optional) attribute

```
update_np_attr_list.da = ['rn', 'ty', 'ri', 'pi', 'ct', 'lt', 'st']
```

Update NP (Not Present) attribute

```
update_m_attr_list.da = [];
```

Update M (Madatory) attribute

```
update_opt_attr_list.da = ['acpi', 'et', 'lbl', 'cr']
```

Update O (Optional) attribute

3.4 create_action()

```
function create_action(request, response, callback) {
  ...
  else if (ty == '50'){ // dataAugment resource
        db_sql.insert_da(request.connection, resource_Obj[rootnm], functaion (err, results) {
          if (!err) {
            callback('200');
          }
          else {
            if (result.code == 'ER_DUP_ENTRY') {
              callback('409-5');
            }
            else {
              console.log('[create_action] create resource error ======== ' + results.code);
              callback('500-4');
            }
          }
        });
      }
  ...
}
```

create_action()에 dataAugment 리소스에 대한 분기문을 추가해줍니다.

`db_sql`로 로드된 sql_action.js의 `insert_da()`를 호출하는 코드를 작성합니다.

3.5 build_resource()

```
function build_resource(request, response, callback) {
  ...
  case '50':
              da.build_da(request, response, resource_Obj, body_Obj, function (code) {
                  callback(code);
              });
              break;
  ...
}
```

`build_resource()`에 dataAugment 리소스에 대한 분기문을 추가해줍니다.

da.js의 `build_da()`를 호출하는 코드를 작성합니다.

3.6 update_action()

```
function update_action(request, response, callback) {
  ...
  else if (ty == '50') {
          db_sql.update_da(request.connection, resource_Obj[rootnm], function (err, results) {
              if (!err) {
                  callback('200');
              }
              else {
                  callback('500-1');
              }
          });
      }
  ...
}
```

`update_action()`에 dataAugment 리소스에 대한 분기문을 추가해줍니다.

da.js의 `update_da()`를 호출하는 코드를 작성합니다.

4. responder.js

4.1 typeRsrc

```
var typeRsrc = {
    "1": "acp",
    "2": "ae",
    "3": "cnt",
    "4": "cin",
    "5": "cb",
    "9": "grp",
    "10": "lcp",
    "13": "mgo",
    "14": "nod",
    "16": "csr",
    "17": "req",
    "23": "sub",
    "24": "smd",
    "27": "mms",
    "28": "fcnt",
    "29": "ts",
    "30": "tsi",
    "38": "tm",
    "39": "tr",
    "50": "da", // here
    "91": "hd_brigs",
    "92": "hd_color",
    "93": "hd_colSn",
    "94": "hd_fauDn",
    "95": "hd_binSh",
    "96": "hd_tempe",
    "97": "hd_bat",
    "98": "hd_dooLk",
    "99": "rsp"
};
```

`typeRsrc`에 dataAugment 리소스에 대한 ty 번호와 resource short name을 정의해줍니다.

4.2 attrLname

```
var attrLname = {
    ...
    "augty":"augmentType",
    "srsrc":"sourceResource",
    "augprm":"augmentParameter",
    "trgrsrc":"targetResource"
};
```

`attrLname`에 dataAugment 리소스의 attribute들을 정의해줍니다.

{attrSname}:{attrLname}의 형태로 attribute short name과 attribute long name을 추가해줍니다.

4.3 attrSname

```
var attriSname = {
    ...
    "augmentType":"augty",
    "sourceResource":"srsrc",
    "augmentParameter":"augprm",
    "targetResource":"trgrsrc"
};
```

마찬가지로 `attrSname`에 dataAugment 리소스의 attribute들을 정의해줍니다.

{attrLname}:{attrSname}의 형태로 attribute short name과 attribute long name을 추가해줍니다.

5. sql_action.js

5.1 insert_da

```
exports.insert_da = function(connection, obj, callback) {
    console.time('insert_da ' + obj.ri);
    _this.insert_lookup(connection, obj, function (err, results) {
        if(!err) {
            var sql = util.format('insert into da (ri, pi, augty, srsrc, augprm, trgrsrc, cr)' + 'value (\'%s\', \'%s\', \'%s\', \'%s\', \'
            db.getResult(sql, connection, function (err, results) {
                if(!err) {
                    console.timeEnd('insert_da ' + obj.ri);
                    callback(err, results);
                }
                else {
                    sql = util.format("delete from lookup where ri = \'%s\'", obj.ri);
                    db.getResult(sql, connection, function () {
                        callback(err, results);
                    });
                }
            });
        }
        else {
            callback(err, results);
        }
    });
};
```

resource.js에서 호출할 export 함수, `insert_da`를 정의해줍니다.

`ri`, `pi`, {M attribute}, `cr`을 mobiusdb에 insert하도록 query를 생성하는 함수입니다.

`ri`, `pi`는 Update, Retrieve 동작시 리소스 탐색에 사용되는 값이므로 DB에 항상 존재해야 합니다.

`cr`은 Delete 동작시 Access permission 검사에 사용되므로 해당 값이 DB에 없다면 리소스 삭제가 불가능합니다.

5.2 update_da

```
exports.update_da = function (connection, obj, callback) {
    console.time('update_da ' + obj.ri);
    _this.update_lookup(connection, obj, function (err, results) {
        if (!err) {
            var sql2 = util.format('update da set augty = \'%s\', srsrc = \'%s\', augprm = \'%s\', trgrsrc = \'%s\' where ri = \'%s\'',
                obj.augty, obj.srsrc, obj.augprm, obj.trgrsrc, obj.ri);
            db.getResult(sql2, connection, function (err, results) {
                if (!err) {
                    console.timeEnd('update_da ' + obj.ri);
                    callback(err, results);
                }
                else {
                    callback(err, results);
                }
            });
        }
        else {
            callback(err, results);
        }
    });
};
```

resource.js에서 호출할 export 함수, `update_da`를 정의해줍니다.

M attribute를 수정하는 query 생성 함수입니다.

6. da.js

```
var url = require('url');
var xml2js = require('xml2js');
var xmlbuilder = require('xmlbuilder');
var util = require('util');
var responder = require('./responder');


exports.build_da = function(request, response, resource_Obj, body_Obj, callback) {
    var rootnm = request.headers.rootnm;

    // body
    // - specific attributes

    if(rootnm == 'da') {
      console.log('testing.da');
      resource_Obj[rootnm].cr = (body_Obj[rootnm].cr) ? body_Obj[rootnm].cr : request.headers['x-m2m-origin'];
    }

    else {
        callback('400-54');
        return;
    }

    request.resourceObj = JSON.parse(JSON.stringify(resource_Obj));
    resource_Obj = null;

    callback('200');
};
```

resource.js로 부터 전달 받은 데이터를 가공하는 모듈입니다.

resource.js에서 호출하는 export 함수 `build_da`를 정의해줍니다.

`resource_Obj[rootnm].cr`값이 request의 `x-m2m-origin`이 되도록 expression을 추가합니다.

7. Create table

Mobius를 설치할때 실행했던 mysql script에는 새로 추가하고자 하는 리소스에 대한 table 내용이 없습니다. 즉, mysql script를 수정하거나 새로 추가하고자 하는 리소스 table을 mobiusdb에 추가해야 합니다.

```
POST : /Mobius
get_resource_from_url (IqbSc_XXd) - /Mobius: 2.038ms
security.check - 4I_kVgCwAz: 0.047ms
testing.da
insert_da /Mobius/test_da
insert into da (ri) value ('/Mobius/test_da')
/home/fabu1ous/Desktop/mobius-dataAug/node_modules/mysql/lib/protocol/Parser.js:
437
      throw err; // Rethrow non-MySQL errors
      ^
```

request를 보냈을 때 error code가 `non-MySQL error`일 경우 위에 설명한대로 새로 추가한 리소스에 대한 table이 없어서 발생하는 것일 확률이 높습니다.

```
use mobius
alter table da add (pi varchar(1024), augty varchar(1024), srsrc varchar(1024), augprm varchar(1024), trgrsrc varchar(1024), cr varchar(102
```

위 sql문을 통해 table을 추가해줍니다.

```
mysql> desc da;
+---------+---------------+------+-----+---------+-------+
| Field   | Type          | Null | Key | Default | Extra |
+---------+---------------+------+-----+---------+-------+
| ri      | varchar(1024) | NO   |     | NULL    |       |
| pi      | varchar(1024) | NO   |     | NULL    |       |
| augty   | varchar(1024) | NO   |     | NULL    |       |
| srsrc   | varchar(1024) | NO   |     | NULL    |       |
| augprm  | varchar(1024) | NO   |     | NULL    |       |
| trgrsrc | varchar(1024) | NO   |     | NULL    |       |
| cr      | varchar(1024) | NO   |     | NULL    |       |
+---------+---------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

8. python test

```
import requests

url = "http://127.0.0.1:7579/Mobius"

def create_ae():
  payload = {
    "m2m:ae": {
      'rn': 'test_dataAugment',
      "api": "0.2.481.2.0001.001.000111",
      "lbl": ["test", "dataAug", "ae"],
      "rr": True,
    }
  }
  headers = {
    'Accept': 'application/json',
    'X-M2M-RI': '12345',
    'X-M2M-Origin': 'StRhqtVBB8V',
    'Content-Type': 'application/json;ty=2'
  }
  response = requests.post(url, json=payload, headers=headers)
  print(response.text)

def create_da():
  payload= {
    "m2m:da": {
      "rn": "test_da",
      "augty":"rotate",
        "srsrc":"test1",
        "augprm":"test2",
        "trgrsrc":"test3"
     }
  }
  headers = {
    'Accept': 'application/json',
    'X-M2M-RI': '12345',
    'X-M2M-Origin': 'StRhqtVBB8V',
    'Content-Type': 'application/vnd.onem2m-res+json; ty=50'
  }
  response = requests.post(url+'/test_dataAugment', json=payload, headers=headers)
  print(response.text)

def retrieve_da():
  payload={}
  headers = {
    'Accept': 'application/json',
    'X-M2M-RI': '12345',
    'X-M2M-Origin': 'StRhqtVBB8V'
  }
  response = requests.get(url+'/test_dataAugment/test_da', json=payload, headers=headers)
  print(response.text)

def delete_da():
  payload={}
  headers = {
    'Accept': 'application/json',
    'X-M2M-RI': '12345',
    'X-M2M-Origin': 'StRhqtVBB8V'
  }
  response = requests.delete(url+'/test_dataAugment/test_da?rcn=0', json=payload, headers=headers)
```

```
  print(response.text)

create_ae()
create_da()
retrieve_da()
delete_da()
retrieve_da()
```


```
{"m2m:da":{"rn":"test_da","ty":50,"pi":"StRhqtVBB8V","ri":"50-20211027033627530089","ct":
"20211027T033627","lt":"20211027T033627","st":0,"et":"20231027T033627","augty":"rotate","
srsrc":"sourceResource","augprm":"augmentParameter","trgrsrc":"targetResource","cr":"StRh
qtVBB8V"}}
{"m2m:da":{"pi":"StRhqtVBB8V","ri":"50-20211027033627530089","ty":50,"ct":"20211027T03362
7","st":0,"rn":"test_da","lt":"20211027T033627","et":"20231027T033627","augty":"rotate","
srsrc":"sourceResource","augprm":"augmentParameter","trgrsrc":"targetResource","cr":"StRh
qtVBB8V"}}
{"m2m:da":{"pi":"StRhqtVBB8V","ri":"50-20211027033627530089","ty":50,"ct":"20211027T03362
7","rn":"test_da","lt":"20211027T033627","et":"20231027T033627","augty":"rotate","srsrc":
"sourceResource","augprm":"augmentParameter","trgrsrc":"targetResource","cr":"StRhqtVBB8V
"}}
{"m2m:dbg":"resource does not exist (get_target_url)"}
```


dataAugment 리소스 Creaet, Retrieve, Delete 테스트